



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Universidad de Antioquia
Facultad de Ingeniería
Ingeniería de Telecomunicaciones – Región

Documentación Técnica

Desafío I: Sweet Crush

Asignatura: Informática II
Profesor: Augusto Enrique Salazar Jiménez
Semestre: 2026-2
Estudiantes: Yudis Paola Apellido Jiménez
Oscar Miguel López Peña

Septiembre de 2026

Índice

1. Introducción	3
1.1. Motivación	3
1.2. Restricciones del Desafío	3
2. Arquitectura de Datos y Manipulación a Nivel de Bits	3
2.1. Codificación de Fichas en 3 Bits	3
2.2. Mapeo de Coordenadas a Memoria	4
2.3. Casos de Manipulación de Bits	4
3. Trazabilidad del Desarrollo	5
3.1. Etapa 1 – Tablero: núcleo bitwise	5
3.1.1. Mapeo y manipulación a nivel de bits	5
3.1.2. Función <code>asignarFicha</code>	5
3.1.3. Función <code>obtenerFicha</code>	5
3.2. Etapa 2 – Gestión de memoria y aleatoriedad	6
3.2.1. Asignación dinámica estricta	6
3.2.2. Generador lineal congruencial	6
3.2.3. Abstracción del cálculo de memoria	6
3.2.4. Renderizado físico en binario	6
3.3. Etapa 3 – Combinaciones	6
3.3.1. Detección de combinaciones	7
3.3.2. Manejo de combinaciones simultáneas	7
3.3.3. Eliminación de variables globales	7
3.4. Etapa 4 – Ciclos	8
3.4.1. Gravedad por dos punteros	8
3.4.2. Reabastecimiento de fichas	8
3.5. Etapa 5 – Cascadas	9
3.5.1. Corrección del cálculo de memoria	9
3.5.2. Procesamiento de cascadas	9
3.5.3. Sistema de puntuación	9
3.6. Etapa 6 – Juego completo	9
3.6.1. Modificación estructural	9
3.6.2. Regla del 65 %	10
3.6.3. Cascadas tras redimensionamiento	10
3.6.4. Menú interactivo de 8 opciones	10
4. Modularización del Proyecto	10
4.1. Archivos del proyecto	11
5. Decisiones de Diseño	11
5.1. Elección de tipos de variables	11
5.2. Estructuras de control	12

5.3. Operadores bitwise	12
6. Resultados y Validación	12
6.1. Verificación del empaquetamiento	12
6.2. Validación de la regla del 65 %	13
7. Conclusiones	13
8. Referencias	13

1. Introducción

El presente documento constituye la **Documentación Técnica** del proyecto *Sweet Crush*, desarrollado como parte del Desafío I de la asignatura Informática II (Semestre 2026-2). El objetivo del proyecto es implementar un juego de combinación de fichas en el lenguaje C++, bajo estrictas restricciones de programación estructurada, uso de memoria dinámica y manipulación directa de bits.

1.1 Motivación

Los juegos de combinación tipo *match-3* son un excelente dominio para poner a prueba el manejo de arreglos bidimensionales, detección de patrones y organización de memoria. Sin embargo, el verdadero reto técnico de este proyecto radica en la **representación compacta del tablero**: cada ficha ocupa exactamente **3 bits** en lugar de un byte o un entero, lo cual obliga a manipular el almacenamiento bit a bit y a resolver los casos en que una ficha se reparte entre dos bytes consecutivos.

1.2 Restricciones del Desafío

- Prohibido usar `struct`, `class`, `template`, STL ni `string`.
- Uso obligatorio de punteros, arreglos y memoria dinámica.
- Uso obligatorio de operadores bitwise (AND, OR, NOT, shift izquierdo, shift derecho).
- Programación modular multi-archivo (`.h` y `.cpp`).
- Solución de consola, sin librerías externas.

2. Arquitectura de Datos y Manipulación a Nivel de Bits

2.1 Codificación de Fichas en 3 Bits

Dado que el juego maneja 6 tipos de fichas estándar y dos estados de control, usar un tipo nativo como `int` (32 bits) generaría un desperdicio de aproximadamente **90 %** de la memoria. Por esa razón, se definió una codificación estricta de 3 bits por casilla:

Binario	Decimal	Significado
000	0	Casilla vacía (se usa durante la gravedad)
001	1	Ficha A
010	2	Ficha B
011	3	Ficha C
100	4	Ficha D
101	5	Ficha E
110	6	Ficha F
111	7	Ficha marcada para eliminación

Cuadro 1: Codificación de fichas en 3 bits.

2.2 Mapeo de Coordenadas a Memoria

Para traducir las coordenadas de una matriz 2D (fila, columna) a un desplazamiento continuo de bits, se usan las siguientes fórmulas:

Posición lineal (I):

$$I = \text{fila} \times \text{columnas} + \text{columna}$$

Bit inicial y byte correspondiente:

$$b = I \times 3 \quad \text{byteIdx} = \left\lfloor \frac{b}{8} \right\rfloor \quad \text{offset} = b \bmod 8$$

Cálculo de memoria estricta:

$$\text{bytes necesarios} = \left\lceil \frac{\text{total fichas} \times 3}{8} \right\rceil = \frac{\text{filas} \times \text{columnas} \times 3 + 7}{8}$$

2.3 Casos de Manipulación de Bits

El acceso al buffer se divide en dos escenarios según el valor de **offset**:

- **Caso 1 (offset ≤ 5):** la ficha está completamente contenida dentro de un byte. Se aplica una máscara `0x07` « offset para aislar los 3 bits requeridos.
- »■ **Caso 2 (offset > 5):** la ficha se reparte entre dos bytes consecutivos. Se calculan `bits_primer_byte = 8 - offset` y `bits_segundo_byte = 3 - bits_primer_byte`. La parte menos significativa se escribe en el byte i y la más significativa en el byte $i + 1$.

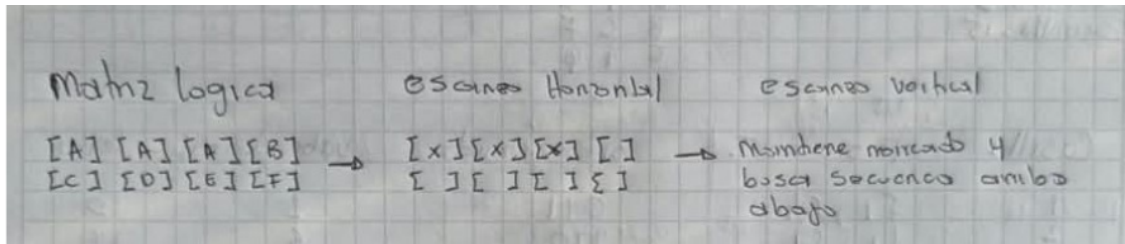


Figura 1: Casos de manipulación de una ficha de 3 bits entre bytes.

3. Trazabilidad del Desarrollo

El proyecto se construyó de forma incremental, evidenciando la evolución desde un prototipo mínimo hasta la solución modular final. A continuación se describen las etapas lógicas del desarrollo.

3.1 Etapa 1 – Tablero: núcleo bitwise

Se implementó el núcleo fundamental de manipulación de memoria y operaciones bitwise (lectura y escritura de 3 bits), un generador pseudoaleatorio sin librerías externas y visualización inicial en formato de caracteres.

3.1.1. Mapeo y manipulación a nivel de bits

La conversión lógica $I = F \times C + c$ al espacio físico se realiza calculando el bit de inicio $b = I \times 3$, el índice del byte $\text{byteIdx} = \lfloor b/8 \rfloor$ y el desplazamiento $\text{offset} = b \bmod 8$.

3.1.2. Función `asignarFicha`

- Caso 1: la ficha cabe en un byte. Se crea la máscara $m = 0x07$ « offset para limpiar los 3 bits destino con AND NOT m y se inserta el valor desplazado con OR (valor « offset).
- Caso 2: la ficha cruza dos bytes. Se calculan $\text{bitsPrimero} = 8 - \text{offset}$ y $\text{bitsSegundo} = 3 - \text{bitsPrimero}$. Los bits menos significativos se ubican en el byte actual y los restantes se desplazan al byte contiguo.

»3.1.3. Función `obtenerFicha`

- »o Caso 1: extrae el valor con $(\text{buffer}[\text{byteIdx}] \gg \text{offset}) \text{ AND } 0x07$.

- »● Caso 2: lee la porción del primer byte, extrae los bits restantes del siguiente, los desplaza bitsPrimero a la izquierda y los recombina con OR.

3.2 Etapa 2 – Gestión de memoria y aleatoriedad

3.2.1. Asignación dinámica estricta

La función `crearTablero` reserva exactamente los bytes requeridos con la fórmula:

$$\text{bytesTot} = \frac{3 \times \text{totalPos} + 7}{8}$$

Se usa `new unsigned char[bytesTot]()` para inicializar el bloque en cero antes del empaquetamiento, y se libera con `delete[]` al terminar el programa.

3.2.2. Generador lineal congruencial

Para evitar librerías de C y cumplir la restricción de distribución uniforme, se implementó un LCG propio:

$$X_{n+1} = (1103515245 \times X_n + 12345) \bmod 2^{31}$$

3.2.3. Abstracción del cálculo de memoria

Se extrajo la fórmula del cálculo dimensional a una función independiente, lo que mejora la modularidad y prepara el sistema para las operaciones de redimensionamiento.

3.2.4. Renderizado físico en binario

Se agregó `mostrarBinario`, que recorre el arreglo de bytes y muestra bit a bit el contenido. Esto permite verificar en tiempo de ejecución que el empaquetamiento de 3 bits no deja huecos de alineación.

3.3 Etapa 3 – Combinaciones

El proyecto pasa de la representación en memoria a la lógica del juego. Se eliminan las variables globales y se pasa a un modelo modular por paso de parámetros.

3.3.1. Detección de combinaciones

Se implementó la función `detectarCombinaciones`, que identifica secuencias de 3 o más fichas idénticas consecutivas en horizontal y vertical.

3.3.2. Manejo de combinaciones simultáneas

Para cumplir con la regla de que una ficha puede pertenecer a una racha horizontal y vertical al mismo tiempo (cruce en T o L), se implementó un algoritmo en dos fases:

1. Escaneo desacoplado: se evalúa el tablero con un arreglo auxiliar `bool* marcados` de tamaño $F \times C$. Si se encuentra una racha ≥ 3 , se marcan las posiciones como `true` sin modificar el tablero original.
2. Marcado físico: una vez terminada la inspección horizontal y vertical, las posiciones marcadas se sobrescriben con el valor 7 (111 binario, representado como X).

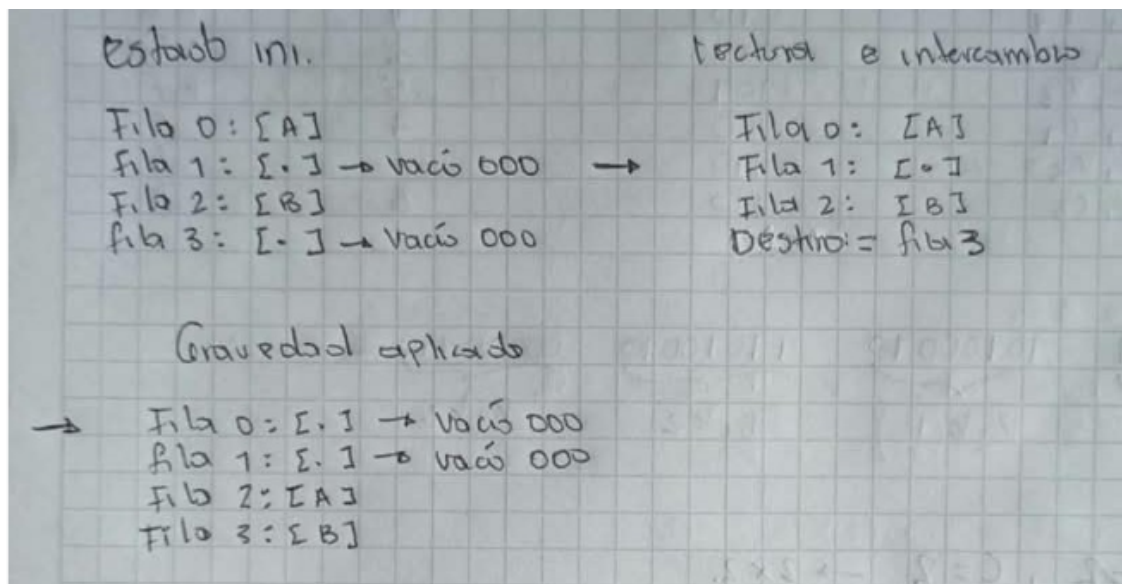


Figura 2: Esquema del algoritmo de detección en dos fases.

3.3.3. Eliminación de variables globales

Se reemplazaron las variables globales por parámetros pasados por referencia y por puntero:

- `unsigned char* & buffer` para modificar la dirección reservada en el heap desde main.

- `const unsigned char*` para las funciones de lectura y renderizado.
- `unsigned int& semilla` para transferir explícitamente el estado del generador.

3.4 Etapa 4 – Ciclos

Se completa la dinámica física del tablero: eliminación de coincidencias, compactación por gravedad y reabastecimiento de nuevas fichas.

3.4.1. Gravedad por dos punteros

Se recorre cada columna de abajo hacia arriba con un puntero destino (inicia en la última fila) y un puntero f (lectura). Cuando `obtenerFicha` entrega un valor distinto de 0, se escribe en `pos_destino`. Si `pos_lectura` es distinto de `pos_destino`, la posición original se sobrescribe con 0 y se decrementa destino.

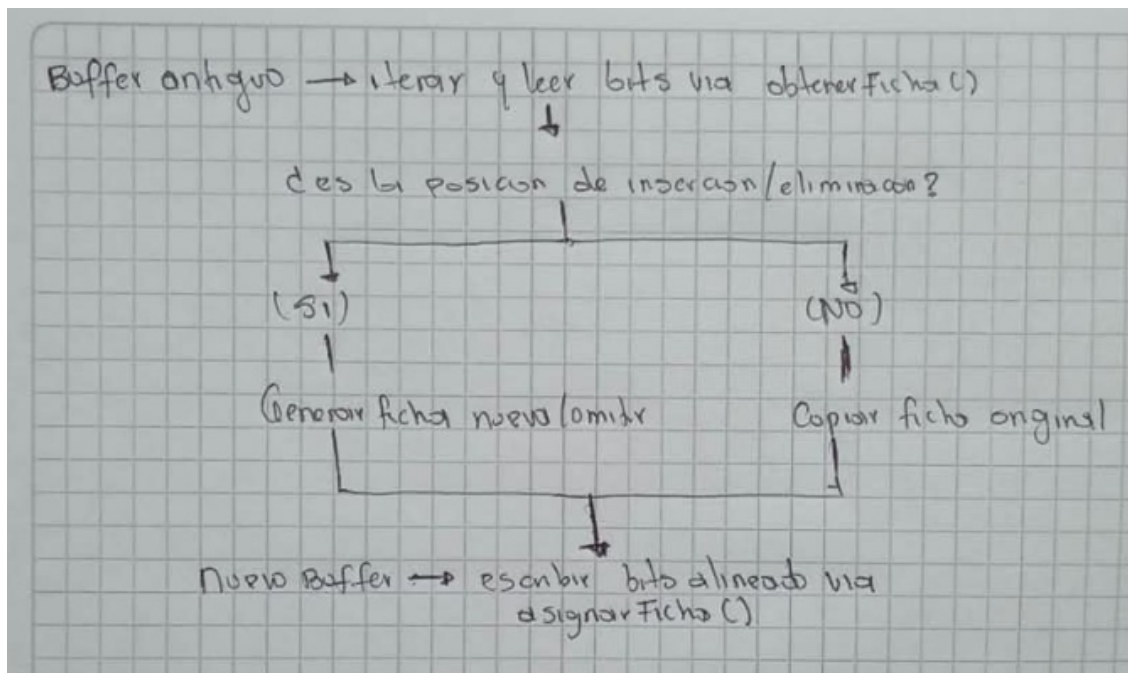


Figura 3: Algoritmo de gravedad por dos punteros.

3.4.2. Reabastecimiento de fichas

Los huecos acumulados en la parte superior de cada columna se rellenan con valores aleatorios generados con `siguienteAleatorio(semilla)`, garantizando que el tablero vuelva a tener $F \times C$ fichas activas.

3.5 Etapa 5 – Cascadas

El proyecto evoluciona a un sistema de juego interactivo con bucles de eventos, cascadas y sistema de puntuación.

3.5.1. Corrección del cálculo de memoria

Se eliminó el margen arbitrario de +2 bytes que se había agregado por seguridad. La fórmula final es la de redondeo superior, que cubre exactamente los bytes requeridos sin desperdicio.

3.5.2. Procesamiento de cascadas

La función procesarCascada contiene un bucle while que repite detección, eliminación, gravedad y relleno hasta que un ciclo completo no produzca nuevas combinaciones. El arreglo bool* marcados se crea y se libera dentro del ciclo para evitar fugas.

3.5.3. Sistema de puntuación

Se otorgan 10 puntos por cada ficha destruida y se lleva un contador acumulativo de las reacciones en cadena producidas durante la partida.

3.6 Etapa 6 – Juego completo

El prototipo pasa a ser un sistema dinámico con reestructuración de dimensiones en tiempo de ejecución.

3.6.1. Modificación estructural

Se implementaron las funciones agregarFila, eliminarFila, agregarColumna y eliminarColumna, que permiten insertar o remover elementos en cualquier posición.

Lógica de transportación de bits: al insertar una fila intermedia, los índices lineales de las fichas cambian completamente. El código lee las fichas del buffer antiguo con obtenerFicha, genera las nuevas con el LCG en la posición insertada y reescribe los bits en un nuevo buffer dimensionado exactamente.

3.6.2. Regla del 65 %

Se añadió un mecanismo de contracción de memoria optimizada:

$$\text{Porcentaje de uso} = \frac{\text{Bytes necesarios}}{\text{Bytes anteriores}} < 0,65$$

- Si el porcentaje es menor a 0.65, se reserva un nuevo buffer, se migran los bits y se libera el anterior con delete[].
- Si es mayor o igual a 0.65, se conserva el bloque actual para evitar operaciones costosas de asignación y liberación.

3.6.3. Cascadas tras redimensionamiento

Las cuatro funciones estructurales activan automáticamente procesarCascada tras reestructurar el tablero, resolviendo cualquier alineación espontánea que se forme.

3.6.4. Menú interactivo de 8 opciones

1. Eliminar ficha manual y detonar cascadas.
2. Agregar fila en cualquier posición.
3. Eliminar fila (mínimo 3 filas).
4. Agregar columna en cualquier posición.
5. Eliminar columna (mínimo 3 columnas).
6. Visualización del tablero y volcado de memoria en binario.
7. Reporte estadístico completo.
8. Salida limpia de memoria.

4. Modularización del Proyecto

La solución final se organiza en seis módulos con responsabilidades claras, cumpliendo con el requisito del desafío de programación modular multi-archivo.

Módulo	Responsabilidad
bits	Generación pseudoaleatoria y manipulación de los 3 bits por ficha
tablero	Creación, liberación y visualización del tablero
juego	Detección de combinaciones, gravedad, relleno y cascadas
estructura	Modificación de filas y columnas, regla del 65 %
estadisticas	Presentación de contadores del juego
main	Menú interactivo y orquestación del flujo principal

Cuadro 2: Módulos del proyecto.

Ventajas frente a un solo archivo: cada módulo se puede probar y modificar por separado; los cambios en uno no obligan a tocar los demás; el código es más fácil de leer y mantener.

4.1 Archivos del proyecto

```

1 src/sweetCrush/
2   sweetCrush.pro
3   main.cpp
4   bits.h      bits.cpp
5   tablero.h   tablero.cpp
6   juego.h     juego.cpp
7   estructura.h estructura.cpp
8   estadisticas.h estadisticas.cpp

```

Listing 1: Estructura de archivos del proyecto

5. Decisiones de Diseño

5.1 Elección de tipos de variables

Variable	Justificación
unsigned char*	Tipo mínimo para almacenar bytes, sin signo, manipulable con bitwise
int	Tamaño suficiente para índices, dimensiones y contadores
unsigned int	Desbordamiento bien definido en el LCG
bool*	Expresa sí/no con 1 byte por posición

Cuadro 3: Tipos de variables utilizados.

5.2 Estructuras de control

- while para secuencias: condición de parada dinámica en la detección de combinaciones.
- for descendente en gravedad: evita pisar fichas no procesadas.
- while (hay combinaciones): el número de cascadas es dinámico.
- do...while en el menú: se ejecuta al menos una vez.
- if / else if para validar coordenadas: evita accesos fuera del buffer.

5.3 Operadores bitwise

Operador	Uso	Ventaja
AND	Enmascarar bits	Aísla los 3 bits sin afectar los demás
OR	Combinar partes de dos bytes	Une información repartida entre bytes
NOT	Limpiar bits antes de escribir	Borra los 3 bits destino sin tocar el resto
Shift izq.	Desplazar el valor al offset	Coloca la ficha en su posición dentro del byte
Shift der.	Extraer bits del byte	Trae la ficha al bit 0 para leerla

Cuadro 4: Operadores bitwise usados.

6. Resultados y Validación

6.1 Verificación del empaquetamiento

La función `mostrarBinario` permite verificar que el tablero se almacena sin huecos de alineación. Para un tablero de 10×10 :

- Posiciones: 100
- Bits: $100 \times 3 = 300$
- Bytes: $\lceil 300/8 \rceil = 38$ bytes

El programa muestra exactamente 38 bytes, confirmando que el empaquetamiento es correcto.

6.2 Validación de la regla del 65 %

En pruebas con un tablero inicial de 12×12 (54 bytes), al eliminar filas progresivamente:

Dimensiones	Bytes	Uso vs. anterior	¿Reduce?
12x12	54	–	–
11x12	50	93 %	No
10x12	45	83 %	No
9x12	41	76 %	No
8x12	36	67 %	No
7x12	32	59 %	Sí

Cuadro 5: Comportamiento de la regla del 65 %.

El redimensionamiento ocurre cuando el uso cae por debajo del 65%, tal como se especifica en el desafío.

7. Conclusiones

- Eficiencia en memoria: se logró un ahorro significativo al almacenar hasta 2 fichas por byte, aprovechando al máximo el heap frente a estructuras tradicionales de arreglos de enteros.
- Robustez algorítmica: la separación en dos fases de la detección de patrones evitó falsos negativos en combinaciones complejas (cruces en T y L).
- Gestión de recursos: la regla del 65% equilibra la conservación de memoria con el rendimiento, reduciendo la fragmentación y las reasignaciones innecesarias.
- Modularidad: la división en seis archivos permite mantenimiento y pruebas independientes, cumpliendo con el requisito del desafío.
- Restricciones cumplidas: no se usaron struct, class, template, STL ni string. Toda la lógica se apoya en punteros, arreglos, memoria dinámica y operadores bitwise.

8. Referencias

- Documento del Desafío I - Sweet Crush, Informática II, Semestre 2026-2.

- Material de estudio de Informática II - Universidad de Antioquia.
- Documentación de operadores bitwise en C++.
- Repositorio del proyecto:
<https://github.com/Oskar-Lopez/Challenge-1-Sweet-Crush>
- Video de demostración:
<https://www.youtube.com/watch?v=a7a47vmc9yA>